

```

import numpy as np
import pandas as pd
import re
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f
from collections import Counter
import matplotlib.pyplot as plt

```

```

df = pd.read_csv("SMSSpamCollection", sep='\t', names=["label", "message"])
df.head()

```

	label	message	
0	ham	Go until jurong point, crazy.. Available only ...	
1	ham	Ok lar... Joking wif u oni...	
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	
3	ham	U dun say so early hor... U c already then say...	
4	ham	Nah I don't think he goes to usf, he lives aro...	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```

print("Number of samples:", len(df))
print("Number of classes:", df['label'].nunique())
print(df['label'].value_counts())

```

```

Number of samples: 5572
Number of classes: 2
label
ham      4825
spam      747
Name: count, dtype: int64

```

```

def clean_text(text):
    text = text.lower()
    text = re.sub(r'^a-zA-Z\s', '', text)
    return text

```

```

df['message'] = df['message'].apply(clean_text)

```

```
df['tokens'] = df['message'].apply(lambda x: x.split())
```

```
df['label'] = df['label'].map({'ham':0, 'spam':1})
```

```
counter = Counter()
for tokens in df['tokens']:
    counter.update(tokens)

vocab = {word:i+2 for i,(word,_) in enumerate(counter.items())}
vocab["<PAD>"] = 0
vocab["<UNK>"] = 1

vocab_size = len(vocab)
```

```
# Download GloVe embeddings if not already present
# The file glove.6B.100d.txt is usually inside glove.6B.zip
import os
if not os.path.exists("glove.6B.100d.txt"):
    print("Downloading GloVe embeddings...")
    !wget -q --show-progress http://nlp.stanford.edu/data/glove.6B.zip
    !unzip -q glove.6B.zip
    print("GloVe embeddings downloaded and unzipped.")
else:
    print("GloVe embeddings already present.")

embedding_dim = 100
embeddings_index = {}

try:
    with open("glove.6B.100d.txt", encoding='utf8') as f:
        for line in f:
            values = line.split()
            word = values[0]
            vector = np.asarray(values[1:], dtype='float32')
            embeddings_index[word] = vector
    print(f"Loaded {len(embeddings_index)} word vectors from glove.6B.100d.t
except FileNotFoundError:
    print("Error: glove.6B.100d.txt not found even after attempted download.
except Exception as e:
    print(f"An unexpected error occurred while loading embeddings: {e}")
```

```
Downloading GloVe embeddings...
glove.6B.zip      100%[=====>] 822.24M  4.93MB/s   in 2m 47s
GloVe embeddings downloaded and unzipped.
Loaded 400000 word vectors from glove.6B.100d.txt
```

```
X = df['tokens']
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```
max_len = 50

def encode(tokens):
    encoded = [vocab.get(word, vocab["<UNK>"]) for word in tokens]
    if len(encoded) < max_len:
        encoded += [vocab["<PAD>"]] * (max_len - len(encoded))
    else:
        encoded = encoded[:max_len]
    return encoded
```

```
class TextDataset(Dataset):
    def __init__(self, X, y):
        self.X = [encode(tokens) for tokens in X]
        self.y = list(y)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return torch.tensor(self.X[idx]), torch.tensor(self.y[idx])
```

```
train_dataset = TextDataset(X_train, y_train)
test_dataset = TextDataset(X_test, y_test)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32)
```

```
class TextCNN(nn.Module):
    def __init__(self, vocab_size, embedding_dim, embedding_matrix):
        super(TextCNN, self).__init__()

        self.embedding = nn.Embedding.from_pretrained(
            torch.tensor(embedding_matrix, dtype=torch.float32),
            freeze=False
        )

        self.conv1 = nn.Conv1d(embedding_dim, 100, kernel_size=3)
        self.relu = nn.ReLU()
        self.pool = nn.MaxPool1d(kernel_size=2)
        self.fc = nn.Linear(100*24, 2)

    def forward(self, x):
```

```

x = self.embedding(x)
x = x.permute(0, 2, 1)
x = self.pool(self.relu(self.conv1(x)))
x = x.view(x.size(0), -1)
x = self.fc(x)
return x

```

```

embedding_matrix = np.zeros((vocab_size, embedding_dim))
for word, i in vocab.items():
    embedding_vector = embeddings_index.get(word)
    if embedding_vector is not None:
        embedding_matrix[i] = embedding_vector

model = TextCNN(vocab_size, embedding_dim, embedding_matrix)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

```

```

epochs = 5

for epoch in range(epochs):
    model.train()
    total_loss = 0

    for inputs, labels in train_loader:
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch {epoch+1}, Loss: {total_loss/len(train_loader)}")

```

```

Epoch 1, Loss: 0.17583101276707436
Epoch 2, Loss: 0.05518831347248384
Epoch 3, Loss: 0.023802208559105306
Epoch 4, Loss: 0.009209228395151772
Epoch 5, Loss: 0.004155525172661457

```

```

model.eval()
all_preds = []
all_labels = []

with torch.no_grad():
    for inputs, labels in test_loader:
        outputs = model(inputs)
        preds = torch.argmax(outputs, dim=1)
        all_preds.extend(preds.numpy())
        all_labels.extend(labels.numpy())

print("Accuracy: ", accuracy_score(all_labels, all_preds))

```

```
print("Precision:", precision_score(all_labels, all_preds))  
print("Recall:", recall_score(all_labels, all_preds))  
print("F1 Score:", f1_score(all_labels, all_preds))  
print("Confusion Matrix:\n", confusion_matrix(all_labels, all_preds))
```

```
Accuracy: 0.9847533632286996  
Precision: 0.9647887323943662  
Recall: 0.9194630872483222  
F1 Score: 0.9415807560137457  
Confusion Matrix:  
[[961   5]  
 [ 12 137]]
```